Pràctiques de Laboratori
Exercicis Pràctica 1 (P1)

Composicions Seqüencials i alternatives, Enums i Iteratives

Assignatura Introducció a la Programació,

Curs 26-27
Primer curs Grau d'Enginyeria Informàtica



UNIVERSITAT DE BARCELONA

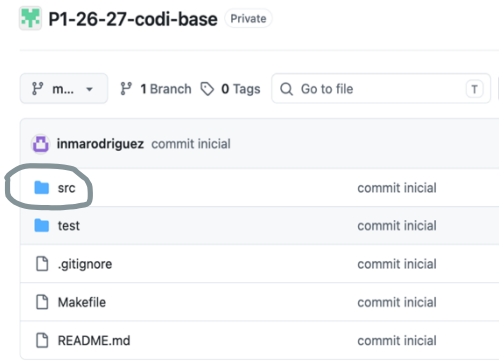
Índex

1. Introducció 3
2. Concepte de programa: funció principal i altres 3
3. Desenvolupament amb TDD 3
4. Exercici resolt: implementar una funció seguint TDD 4
5. Exercici resolt: cridar la funció amb entrada/sortida per teclat/pantalla 8
6. Exercicis a resoldre 8
   6.1. Exercicis de seqüencials i alternatives 9
   6.2. Exercicis de enums 15
   6.3. Exercicis de iteratives 16

1. Introducció

Aquest document te dos finalitats, per un costat introduir la tècnica de desenvolupament que seguirem a l'assignatura (TDD), i per altre costat donar-vos una primera llista d'exercicis. Feu si us plau tots els que pugueu, encara que per la prova P1 us sol·licitarem pujar al CV només alguns. Direm quins uns dies abans del lliurament. La prova P1 consistirà en modificar, in-situ al laboratori, un dels exercicis lliurats.

Més endavant us compartirem un *repositori* amb el codi base de la P1. Treballareu tot el temps en el directori `src` que mostra l'imatge de la dreta. Veureu un directori, `test`, NO EL TOQUEU, s'utilitza per passar els tests definits pel professorat. A més, NO heu de modificar el fitxer Makefile.



2. Concepte de programa: funció principal i altres funcions

Recordeu que un programa és un conjunt d'una o més funcions que col·laboren per resoldre un problema. Entre aquestes funcions hi ha la funció principal, **`main()`**, per on comença i acaba l'execució del programa i, des d'on es criden **altres funcions**. Recordeu també que tot programa ha de tenir només una única funció `main()`.

És molt temptador començar a escriure el codi d'aquestes altres funcions immediatament. En aquest curs farem just el contrari: abans d'escriure cap codi, definirem què comportament s'espera de la funció, és a dir, escriurem uns **tests**, i a partir d'aquest moment definirem el codi que fa passar aquests tests. Aquesta metodologia s'anomena TDD (Test-Driven Development), i s'introdueix a la següent secció.

3. Desenvolupament amb TDD

La metodologia TDD estableix que, amb cada funcionalitat que heu de implementar en el vostre programa, se segueix un procés iteratiu de 4 passos, tal i com mostra la

Il·lustració 1. Fixa't que des del Pas 1 fins al pseudocodi del Pas 4 NO depenem del llenguatge de programació.

A més, observa les fletxes. S'itera des del Pas 3 al Pas 4 fins que TOTS els tests passin. I, si afegim una nova funcionalitat, hem d'anar al Pas 1 i seguir el procés de nou.

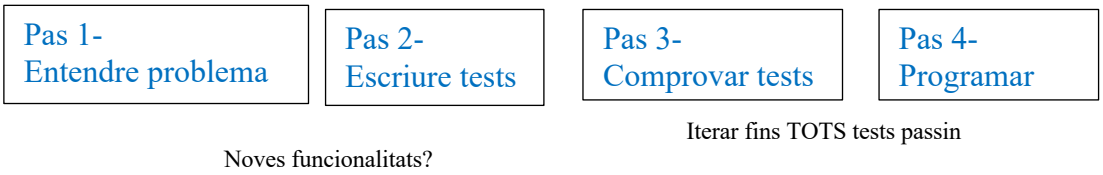
Pas 1. Entendre el problema a resoldre. Què ha de fer la funció que implementi aquesta funcionalitat? **Pensa què ha de fer**. És a dir, quin comportament ha de tenir la funció per a diferents casos de prova (quina sortida esperes per a diferents casos d'entrada).

També s'ha de decidir el nom de la funció, quin/s paràmetres tindrà i què ha de retornar. Cal dir que en aquesta llista d'exercicis us donarem nom, paràmetres i retorn de les funcions, això us donarà coneixement per a pensar-lo més endavant per vosaltres mateixos.

Pas 2. Escriure tests. Escriu el tests (o casos de prova) on, donada una entrada coneguda, es defineix quina és la sortida esperada. Inclou també casos límit o especials.

Pas 3. Comprovar tests. Crida la funció amb els tests del Pas 2. Si un test falla, encara no implementis més funcionalitat de la necessària. Escriu només el codi imprescindible perquè passi aquell test. En la primera iteració sempre falla, ja que la funció està buida!

Pas 4. Programar. **Pensa com** resoldràs el problema. Sempre ajuda escriure el *pseudocodi* (passos a seguir) de la funció amb llapis i paper. A continuació, tradueix el pseudocodi a un llenguatge de programació (en el nostre cas a C), i itera de nou al Pas 3 fins que tots els tests funcionin**.** Si s'escau es pot tornar al Pas 1 per afegir una nova funcionalitat.



Il·lustració 1. Procés del TDD

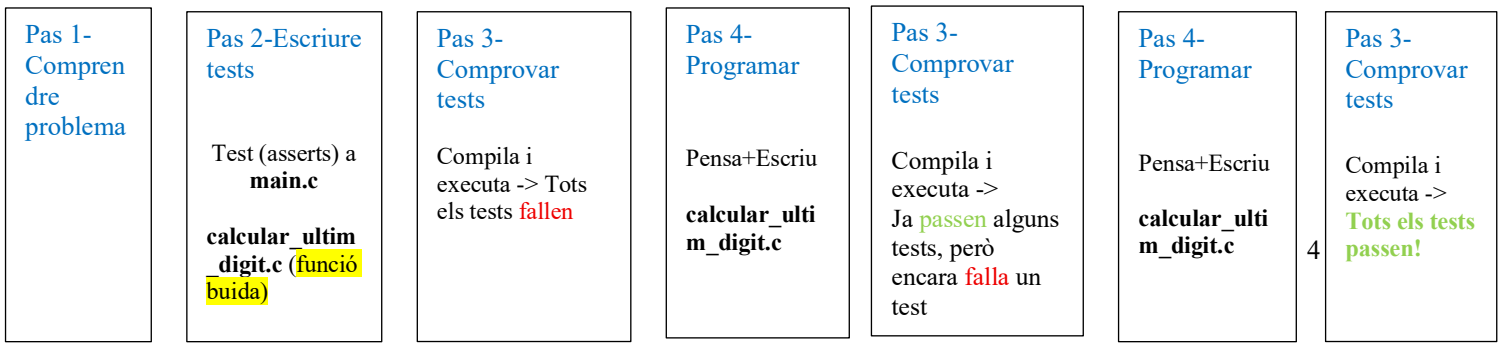
Si us plau, analitzeu l'exemple resolt a continuació per Entendre aquest procés de 4 passos que heu de seguir quan feu els exercicis.

4. Exercici resolt: implementar una funció seguint TDD

Escriu la funció `calcular_ultim_digit()`, que donat un nombre enter, `n`, retorni la seva ultima xifra. (**ultim_digit.c)**

```
int calcular_ultim_digit(int n);
```

La Il·lustració 2 mostra el procés de TDD que seguirem a continuació per a implementar la funció `calcular_ultim_digit()` (amb dues iteracions entre el Pas 3 i Pas 4).



Il·lustració 2. TDD amb la implementació de la funció calcular_ultim_digit()

Pas 1-Entendre el problema

Llegim l'enunciat de l'exercici per a Entendre el problema a resoldre. La funció ha de rebre un nombre enter i retornar-ne l'última xifra. Abans de res, ens fem algunes preguntes: què passa si el nombre només té una xifra? (3 → 3), què passa si el nombre és 0? (0 → 0), què passa si el nombre és molt gran? (78956 → 6), què passa si el nombre és negatiu? (-37 → 7). En aquest últim cas, observem que el signe del nombre no forma part de l'últim dígit, per tant el resultat sempre ha de ser un valor entre 0 i 9.

Amb aquesta anàlisi ja podem definir els tests que comprovaran si la funció es comporta correctament.

Pas 2-Escriure tests:

Un vegada entès el problema, ja podem escriure una Taula de Tests (o casos de prova) que especifiquen quina és la *Sortida esperada* per a diferents valors d'*entrada*. Fixa't com les files de la taula s'escriuen amb la funció `assert()` al fitxer **main.c** de sota. La funció `assert()` comprova que el resultat obtingut coincideixi amb el resultat esperat; en cas contrari, el test falla i el programa s'atura. Per tant, si arribem a l'última línia del `main()` és perquè tots els tests han passat.

Taula de Tests (casos de prova)	
Entrada *n*	**Sortida esperada**
3	3
21	1
78956	6
0	0
-37	7

Casos especials o límit:
0 perquè és el menor enter no negatiu i només té una xifra.
-37 perquè cal definir quin és l'últim dígit d'un nombre negatiu.

ultim_digit.c

```
// Funció que retorna l'últim dígit
int calcular_ultim_digit(int n) {

  //fixeu-vos que la funció està BUIDA, escrivim return per a evitar error de compilació
```

```
  return -1;
}
```

main.c

```
 1 #include <stdio.h>
 2 #include <assert.h>
 3 int calcular_ultim_digit(int n);
 4
 5 int main(void) {
 6
 7
 8       assert(calcular_ultim_digit(3)==3);//comprova que per l'entrada 3 la
sortida és 3
 9       printf("Ha passat calcularUltimDigit(3)==3\n");

10       assert(calcular_ultim_digit(21)==1); //entrada 21, sortida 1
11       printf("Ha passat calcularUltimDigit(21)==1\n");
12
13       assert(calcular_ultim_digit(78956)==6);
14       printf("Ha passat calcularUltimDigit(78956)==6\n");
15
16       assert(calcular_ultim_digit(0)==0);
17       printf("Ha passat calcularUltimDigit(0)==0\n");
18
19
20       assert(calcular_ultim_digit(-37)==7);
21       printf("Ha passat calcularUltimDigit(-37)==7\n");
22
23       printf("TOTS els tests han passat!\n");
24
25       return 0;
26 }
```

Pas 3-Comprovar tests: Compila i executa els fitxers anteriors (amb els `ls` veus que n'hi ha al directori abans i després), observa que ja el primer test falla (assert a la línia 8) perquè el codi de la funció està buit:

```
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % ls
main.c          ultim_digit.c
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % gcc -Wall -std=c17 -pedantic ultim_digit.c main.c -o programa
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % ls
main.c          programa        ultim_digit.c
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % ./programa
Assertion failed: (calcular_ultim_digit(3)==3), function main, file main.c, line 8.
zsh: abort      ./programa
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 %
```

Passem doncs al següent pas on pensem el pseudocodi o lògica que defineix **com ha de comportar-se la funció per passar aquest test**.

Pas 4-Programar: Abans d'escriure el codi, pensa com pots obtenir l'última xifra d'un nombre enter. Observa que el residu de dividir un nombre entre 10 sempre correspon a la seva última xifra (per exemple, 21 % 10 = 1 i 78956 % 10 = 6). Per tant, una primera solució és retornar el resultat de n % 10.

Ara que ja tens la lògica que resol el problema, pots implementar la funció i així aconseguir que els tests que havien fallat al pas anterior passin:

ultim_digit.c

```
int calcular_ultim_digit(int n);

// Funció que retorna l'últim dígit
int calcular_ultim_digit(int n) {
```

```
  return (n % 10);
}
```

Torna al Pas 3-Comprovar tests: La següent imatge mostra com al executar el programa, ara varis test funcionen, però NO funciona el test a la línia 20. Amb un nombre enter negatiu el mòdul és negatiu, i per tant la sortida és -7, que no és igual a la sortida esperada 7 (veure **Taula** dalt):

```
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % ls
main.c          programa        ultim_digit.c
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % gcc -Wall -std=c17 -pedantic ultim_digit.c main.c -o programa
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % ./programa
Ha passat calcularUltimDigit(3)==3
Ha passat calcularUltimDigit(21)==1
Ha passat calcularUltimDigit(78956)==6
Ha passat calcularUltimDigit(0)==0
Assertion failed: (calcular_ultim_digit(-37)==7), function main, file main.c, line 20.
zsh: abort      ./programa
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 %
```

Recomanem què quan tingueu un cas que NO superi un test amb assert, podeu incloure un `print` just abans de l'assert per veure quin valor retorna la funció. D'aquesta manera, podreu entendre per què la prova no s'està superant.

Torna al Pas 4-Programar: Torna al codi per a modificar el codi de **ultim_digit.c** i aconseguir així que passi el test de la línia 20. Per allò, podem calcular el valor absolut de `n` per a convertir-lo a positiu en cas de ésser negatiu, i després fer el mòdul de 10. Observa que per calcular el valor absolut usem la funció `abs()` de la llibreria `stdlib`:

Ultim_digit.c

```c
#include <stdlib.h> // Per usar abs()

int calcular_ultim_digit(int n);

// Funció que retorna l'últim dígit (gestionant negatius)
int calcular_ultim_digit(int n) {

  return abs(n) % 10;
}
```

Torna al Pas 3-Comprovar tests: Ara torna a executar el programa, és a dir, torna a comprovar els tests. La següent figura mostra com tots els `asserts` passen:

```
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % ls
main.c          programa        ultim_digit.c
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % gcc -Wall -std=c17 -pedantic ultim_digit.c main.c -o programa
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % ./programa
Ha passat el test calcularUltimDigit(3)==3
Ha passat el test calcularUltimDigit(21)==1
Ha passat el test calcularUltimDigit(78956)==6
Ha passat el test calcularUltimDigit(0)==0
Ha passat el test calcularUltimDigit(-37)==7
TOTS els tests han passat!
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 %
```

Hem finalitzat el procés d'implementació de `calcular_ultim_digit()`, amb tots els asserts al vostre **main.c** dintre del directori `src`.

Ara ja podeu comprovar els tests definits pel professorat (directori `test`). Per allò, situeu-vos al directori arrel del repositori (`P1-26-27-codi-base`) i executeu "make test_all". A la sortida veureu si el vostre codi passa tots aquests tests.

Per esborrar els fitxers executables que es generen executa a la consola el comando "make clean". Recordeu que NO heu de modificar el fitxer Makefile.

5. Exercici resolt: cridar la funció amb entrada/sortida per teclat/pantalla

Ara que ja funciona correctament la funció `calcular_ultim_digit()`, pots re-escriure el `main()` per a fer entrada **per teclat i sortida per pantalla**. Comenta els asserts i escriu el codi per llegir amb `scanf` i per escriure amb `printf`:

main.c

```c
#include <stdio.h>
#include <assert.h>

int calcular_ultim_digit(int n);

int main(void) {

    /*
     assert(calcular_ultim_digit(3)==3);
     assert(calcular_ultim_digit(21)==1);
     assert(calcular_ultim_digit (78956)==6);
     assert(calcular_ultim_digit (0)==0);
     assert(calcular_ultim_digit (-37)==7);
     printf("Tots els casos de prova han passat!\n");
     */
     int numero;
     printf("Introdueix un nombre enter: ");
     if (scanf("%d", &numero) == 1) {
        printf("L'última xifra és: %d\n", calcular_ultim_digit(numero));
     } else {
        printf("Error: L'entrada no és un nombre enter vàlid.\n");
     }
     return 0;
}
```

Compila i executa:

```
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % gcc -Wall -std=c17 -pedantic ultim_digit.c main.c -o programa
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % ./programa
Introdueix un nombre enter: 234
L'última xifra és: 4
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 % ./programa
Introdueix un nombre enter: hola
Error: L'entrada no és un nombre enter vàlid.
(base) inma@Inmaculadas-MacBook-Pro-2 ex1 %
```

6. Exercicis a resoldre

A continuació trobareu exercicis a resoldre, organitzats en tres subseccions tal i com estan al repositori github de la P1.

En primer lloc, exercicis que es resolen només fent servir composicions seqüencials (sentències una darrera l'altre) i alternatives (if, if-else-if, switch). En segon lloc, exercicis sobre enums. Finalment, exercicis en què cal fer servir iteratives (while, for, do-while).

Tots els exercicis s'han de resoldre **sense utilitzar arrays** ja que encara no els hem introduït.

6.1. Exercicis de seqüencials i alternatives

Per resoldre aquests exercicis, segueix si us plau el procés TDD que s'ha introduït abans. Quan arribis al Pas 4-Programar, recorda parar-te a pensar la lògica, definint el pseudocodi o passos mínims per a fer passar el test que falla. En els primers exercicis us donem pistes de com heu d'enfrontar aquesta etapa de pensar la lògica abans de escriure cap codi C.

Quan la funció que es demana passi tots els tests (asserts) que heu definit al **main.c**, feu add i commit al repositori local, i a continuació push per pujar al repositori remot en github.

Quan tots els tests passin, ja podeu també comentar els asserts del `main()` i, si voleu (no és obligatori en la majoria d'exercicis), fer Entrada per teclat i Sortida per pantalla (E/S).

1. Escriu la funció `calcular_ultim_digit()`, que donat un nombre enter, `n`, retorni la seva ultima xifra. (funció a **ultim_digit.c,** asserts a **main.c)**

   Exercici ja resolt a la secció Exercici resolt: implementar una funció seguint TDD

   ```
   int calcular_ultim_digit(int n)
   ```

2. Escriu una funció `es_digit()` que rep un nombre enter, `n`, i ens diu si és un dígit o no (funció a **digit.c**) **(**asserts a **main.c)**. Per exemple, si el nombre és 4 retorna true, i si és 234 retorna false.

   ```
   bool es_digit(int n)
   ```

   Pensa la lógica de la solució (pseudocodi en llapis i paper, del Pas 4-programar):

   > Un dígit és qualsevol nombre enter comprès entre 0 i 9, ambdós inclosos. Per tant, la funció ha de comprovar si el valor rebut, `n`, compleix aquesta condició, i en cas afirmatiu retornar true, i false en cas contrari. Per tant, es necessita una sentència condicional `si` amb aquesta condició.

   ```
   Funció es_digit(n)

     resultat = false

     Si n >= 0 i n <= 9
         resultat = true
     Fin si

     retornar resultat

   Fi funció
   ```

   NOTA: Pensa si hi ha altres formes de fer-ho i prova-les! Fixa't que tots els tests (`asserts`) haurien de passar amb la nova solució.

Si ja està testejada la funció `es_digit()` amb els asserts corresponents, fes push per pujar al repositori remot a github. Ara ja pots implementar la E/S a la funció `main()` perquè si l'usuari, per exemple, introdueix per teclat 4 surti el missatge "És un dígit", i si introdueix 478 surti el missatge "No és un dígit". Si l'usuari no introdueix un nombre enter, ha de sortir "Error: l'entrada no es un nombre enter vàlid" **(**funció a **ultim_digit.c,** E/S a **main.c)**

3. Escriu la funció `calcular_any_naixement()` que rep l'edat d'una persona el 31 de desembre de 2026, `edat_actual`, i calcula el seu any de naixement. Per exemple, si l'usuari introdueix l'edat 18, el programa donarà com sortida "Vas néixer l'any 2008. La funció assumeix que rep un valor comprès entre 1 i 120, ambdós inclosos. **(**funció a **edat.c)**

```
int calcular_any_naixement(int edat_actual)
```

Escriu altre funció, `calcular_any_doble_edat()`, que rep l'edat d'una persona el 31 de desembre de 2026, `edat_actual`, i calcula en quin any tindrà el doble de l'edat que té el 31 desembre de 2026. Per exemple, si l'usuari introdueix l'edat 18, la funció retornarà 2044.

```
int calcular_any_doble_edat(int edat_actual
```

Pensa primer la lògica de la solució (pseudocodi en llapis i paper):

L'any de naixement es calcula restant a l'any actual l'edat, i el doble d'edat es calcula sumant a l'any actual l'edat actual.

```
Funció calcular_any_naixement(edat)
  retornar 2026 - edat
Fi funció

Funció calcular_any_doble_edat(edat)
  retornar 2026 + edat
Fi funció
```

NOTA: Pensa si hi ha altres formes de fer-ho i prova-les! Fixa't que tots els tests (`asserts`) haurien de passar amb la nova solució.

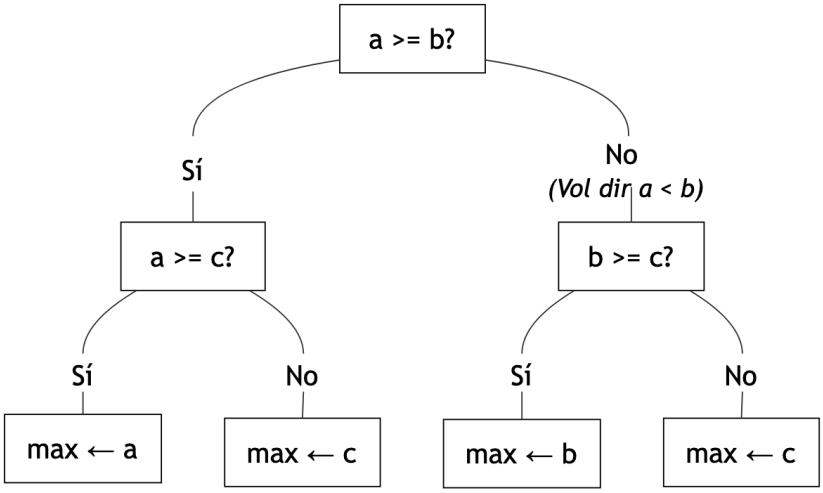
Si ja estan testejades les funcions, fes push per pujar al repositori remot a github. Ara ja pots implementar la funció `main()` perquè l'usuari introdueixi l'edat per teclat i surti el missatge "Vas néixer al xxxx" i "L'any xxx tindràs el doble d'edat". Si l'usuari no introdueix un nombre enter, com ara una cadena de caràcters, ha de sortir "Error: la entrada no és un nombre enter vàlid" **(**funció a **edat.c,** E/S a **main.c)**

4. Escriu la funció `maxof3()`, que rep tres enters, `a`, `b`, `c`, i retorna el màxim del tres. **(**funció a **maxof3.java,** asserts a **main.c)**

```
int maxof3(int a, int b, int c)
```

Pensa primer la lógica de la solució (pseudocodi en llapis i paper):

Fixa't com el dibuix a sota ens ajuda a construir la lógica de la solució. Comparant els dos primers nombres, `a` i `b`. Si `a` és més gran o igual que `b`, només cal decidir si el màxim és `a` o `c` (veure part esquerra de l'arbre a sota). En canvi, si `b` és més gran que `a`, només cal decidir si el màxim és `b` o `c` (veure part dreta de l'arbre a sota). Per tant, primer comparem `a` i `b`, i després comparem el més gran dels dos amb `c`.



Per tant, el pseudocodi és:

```
Funció maxof3(a, b, c)

   max

   Si a >= b entonces
       Si a >= c entonces
           max ← a
       Si no
           max ← c
       FinSi
   Si no
       Si b >= c entonces
           max ← b
       Si no
           max ← c
       FinSi
   FinSi

   retornar max

Fi Funció
```

5. Escriu la funció `max_of3_amb2if()` que calcula el màxim de tres nombres enters `a, b, c`, usant com a molt dues instruccions if. **(**funció **a maxof3Amb2if.c,** asserts a **main.c)**

   ```
   int max_of3_amb2if(int a, int b, int c)
   ```

6. Una circumferència en el pla real està especificada per les coordenades del centre (`x1, y1`) i el radi `r`. Escriu la funció `punt_dins_circumferencia` , que donats el centre i el radi de la circumferència i un punt `(x, y)`, indica si el punt és dins la circumferència o fora. La funció retorna true si el punt està dins (o al límit), false si està fora. **(**funció a **en_circumferencia.c,** asserts a **main.c)**

   ```
   bool punt_dins_circumferencia(double cx, double cy, double r,
                                 double px, double py)
   ```

   Més tard, quan la funció estigui completament testejada, fes la funció `main()` amb E/S. Per exemple, si l'usuari introdueix el centre (0.0, 0.0), el radi 1 i el punt (1.0, 0.0), surt el missatge: "El punt és dins la circumferència.". Si s'introdueix el centre (1.0, 2.0), radi 1.2 i el punt (0.0, 0.0) surt el missatge: "El punt és fora la circumferència.". **(**funció a **en_circumferencia.c,** E/S a **main.c)**

7. Un rectangle amb els costats paral·lels als eixos està especificat per les coordenades del vèrtex inferior esquerre `(x1, y1)` i del vèrtex superior dret `(x2, y2)`. Escriu una funció que, donades aquestes coordenades i les coordenades d'un punt `(x, y)`, retorni true si el punt és a dins del rectangle o sobre la seva vora, i false en cas contrari. **(**funció a **en_rectangle.c,** asserts a **main.c)**

   ```
   bool punt_dins_rectangle(double x1, double y1, double x2, double y2,
                            double px, double py)
   ```

   Més tard, quan la funció estigui completament testejada, fes la funció `main()` amb E/S. Per exemple, si l'usuari introdueix les coordenades del rectangle `(2.5, 2.5)` i `(6.0, 6.0)`, i el punt `(5.0, 4.2)`, es mostrarà el missatge "El punt està dintre del rectangle"**.** Si introdueix el punt `(5.1, 7.0)`, es mostrarà "El punt està fora del rectangle"**. (**funció a **en_rectangle.c,** E/S a **main.c)**

8. Escriu la funció `calcular_min_rectangle()` que donats dos rectangles amb costats paral·lels als eixos calcula el rectangle mínim que els conté. Els paràmetres (`ax1, ay1`) i (`ax2, ay2`) corresponen, respectivament, al vèrtex inferior esquerre i al vèrtex superior dret del primer rectangle. De la mateixa manera, (`bx1, by1`) i (`bx2, by2`) corresponen al segon rectangle.

   La funció ha de calcular el rectangle mínim que conté tots dos rectangles i retornar-ne les coordenades mitjançant els paràmetres de sortida `rx1`, `ry1`, `rx2` i `ry2`, que representen el vèrtex inferior esquerre i el vèrtex superior dret del rectangle resultat. Per exemple, si el primer rectangle té les coordenades (1.0, 3.0) (3.0, 5.0) i el segon les coordenades (2.0, 1.0) (4.0, 4.0), el rectangle mínim que retorna en `rx1`, `ry1`és (1.0, 1.0) i en `rx2`, `ry2` (4.0 , 5.0). **(**funció a **min_rectangle.c,** asserts a **main.c)**

   La funció assumeix que `ax1 <= ax2`, `ay1 <= ay2`, `bx1 <= bx2`, i `by1 <= by2`.

   ```
   void calcular_min_rectangle(double ax1, double ay1,
   ```

```
double ax2, double ay2,
double bx1, double by1,
double bx2, double by2,
double *rx1, double *ry1,
double *rx2, double *ry2)
```

9. Escriu la funció `intersequen_intervals()` que, donats dos intervals [x1, x2] i [y1, y2] descrits pels quatre nombres reals `x1`, `x2`, i `y1`, `y2`, retorna true si aquests intervals s'intersequen (incloent-hi el cas en què comparteixen un extrem), i false en cas contrari. Per exemple, amb els intervals [2.0, 4.4] [5.0, 6.3] ha de retornar false, i amb els intervals [2.0, 4.4] [3.0, 6.3] ha de retornar true. **(**funció a **intervals.c,** asserts a **main.c)**

   La funció assumeix que `x1 <= x2,` i `y1 <= y2.`

   ```
   bool intersequen_intervals(double x1, double x2, double y1, double y2)
   ```

10. Escriu una que generi de forma aleatòria dos nombres d'una xifra, pregunti a l'usuari el resultat de la multiplicació d'aquests nombres, i doni l'enhorabona "Ben fet!" si ho fa bé i si ho fa malament li animi a estudiar més "Has d'estudiar la taula de multiplicar". (generació aleatoris i E/S a **main.c**)

    Escriu una funció `comprova_multiplicacio()` que, donats els noms aleatoris `num1` i `num2`, i la resposta de l'usuari, `resposta_usuari`, faci la comprovació i retorni true si la resposta es correcta, i en cas contrari retorni false.

    ```
    bool comprova_multiplicacio(int num1, int num2, int resposta_usuari);
    ```

    (funció a **multiplica_nombres_aleatoris.java)**

    Per saber com generar un nombre aleatori en C, consulta l'apèndix 7.1

11. Escriu la funció `resol_equacio()`, que resolgui una equació de segon grau de la forma `ax² + bx + c = 0`. La funció ha de retornar 0 si l'equació no té cap solució real, 1 si té una única solució real (arrel doble), que s'ha de retornar a través del paràmetre `sol1`, o 2 si té dues solucions reals, que s'han de retornar a través dels paràmetres `sol1` i `sol2`. La funció assumeix que `a != 0`. **(**funció a **equació_segon_grau.c)**

    ```
    int resol_equacio(double a, double b, double c, double *sol1, double *sol2)
    ```

    Més tard, quan la funció estigui completament testejada, fes la funció `main()` amb E/S perquè demani per teclat els coeficients a, b i c, cridi `resol_equacio()` i mostri el nombre de solucions i els seus valors arrodonits a dues xifres decimals.

    Per exemple, si l'usuari introdueix a=1, b=2, c=4, ha de sortir el missatge: "1.0x^2 + 2.0x + 4.0 = 0 no té solucions reals". Si s'introdueix a=1, b=2, c=1, ha de sortir el missatge: "1.0x^2 + 2.0x + 1.0 = 0 té una única solució x = -1.0". Si s'introdueix a=-4, b=2, c=3, ha de sortir el misssatge: "-4.0x^2 + 2.0x + 3.0 = 0 té com a solucions x1 = -0.65 i x2 = 1.15". (funció a **equació_segon_grau.c,** E/S a **main.c**)

12. Escriu la funció `calcular_preu_menu()`, que, donada una opció de menú, `opcio`, retorni el seu preu. Les opcions disponibles són: 1 (Menú Bàsic, 10,50 €), 2 (Menú Complet, 14,90 €) i 3 (Menú Degustació, 18,00 €). Si l'opció no és vàlida, la funció ha de retornar 0.0. Utilitzeu la sentencia `switch` a la funció. (funció a **menu_migdia.c**)

    ```
    double calcular_preu_menu(int opcio)
    ```

Més tard, quan la funció estigui completament testejada, fes la funció `main()` amb E/S perquè mostri el menú per pantalla, demani a l'usuari l'opció desitjada i el nombre de menús que vol demanar, calculi l'import total de la comanda el mostri amb idues xifres decimals. Si l'opció introduïda no és vàlida, s'ha de mostrar el missatge "Opció no vàlida." (funció a **menu_migdia.c,** E/S a **main.c**).

Si us plau, segueix el mateix format de els exemples d'execució a continuació:

```
(base) inma@Inmaculadas-MacBook-Pro-2 ex12 % ./main
--- MENÚ DEL MIGDIA ---
1. Menú Bàsic (10.50€)
2. Menú Complet (14.90€)
3. Menú Degustació (18.00€)
Escull una opció: Escull una opció: 2
Digues el nombre de menús: 3
Has de pagar: 44.70€
(base) inma@Inmaculadas-MacBook-Pro-2 ex12 % ./main
--- MENÚ DEL MIGDIA ---
1. Menú Bàsic (10.50€)
2. Menú Complet (14.90€)
3. Menú Degustació (18.00€)
Escull una opció: Escull una opció: 5
Opció no vàlida.
(base) inma@Inmaculadas-MacBook-Pro-2 ex12 % ./main
--- MENÚ DEL MIGDIA ---
1. Menú Bàsic (10.50€)
2. Menú Complet (14.90€)
3. Menú Degustació (18.00€)
Escull una opció: Escull una opció: 3
Digues el nombre de menús: hola
(base) inma@Inmaculadas-MacBook-Pro-2 ex12 %
```

13. Amplia la funció `main()` de l'exercici anterior perquè segueixi demanant a l'usuari l'opció desitjada i el nombre de menús que vol demanar, però a més ha de calcular el import de la comanda, ha d'aplicar un **descompte** del 10 % si es demanen 5 o més menús. Finalment, també ha de calcular l'import amb **IVA** del 10 % i mostrar per pantalla el subtotal, el descompte aplicat (si escau), l'import de l'IVA i el total final amb dues xifres decimals. La funció `calcular_preu_menu()` no canvia. (funció a **menu_migdia.c,** E/S a **main.c**).

    Si us plau, segueix el mateix format dels exemples d'execució a continuació:

```
(base) inma@Inmaculadas-MacBook-Pro-2 ex13 % ./main
--- MENÚ DEL MIGDIA ---
1. Menú Bàsic (10.50€)
2. Menú Complet (14.90€)
3. Menú Degustació (18.00€)
Escull una opció: 1
Digues el nombre de menús: 6

Subtotal: 63.00€
Descompte: 6.30€
IVA (10%): 5.67€
Total: 62.37€
```

6.2. Exercicis de enums

14. Escriu la funció `determinar_guanyador()` que ha de determinar el resultat d'una partida del joc Pedra, Paper, Tisores i retornar EMPAT si tots dos jugadors han fet la mateixa jugada, GUANYA_J1 si guanya el jugador 1 o GUANYA_J2 si guanya el jugador 2.

    La funció assumeix que j1 i j2 són valors vàlids de l'enumeració Jugada.

    Defineix els enums i cabecera a **pedra_paper_tisores.h:**

```
typedef enum { PEDRA, PAPER, TISORES } Jugada;
typedef enum { EMPAT, GUANYA_J1, GUANYA_J2 } Resultat;

resultat determinar_guanyador(Jugada j1, Jugada j2);
```

    I el cos de la funció a **pedra_paper_tisores.c**.

    Més tard, quan la funció `determinar_guanyador()` estigui completament testejada, implementeu la funció `main()` en perquè demani a l'usuari una jugada (0 = PEDRA, 1 = PAPER, 2 = TISORES), generi aleatòriament la jugada de la màquina, cridi la funció `determinar_guanyador()` i mostri per pantalla les dues jugades i el resultat de la partida. (funció a **pedra_paper_tisores.h** i **pedra_paper_tisores.c**, E/S a **main.c**)

    Si us plau, segueix el mateix format dels exemples d'execució a continuació:

```
(base) inma@Inmaculadas-MacBook-Pro-2 ex14 % ./main
Joc: 0 = PEDRA, 1 = PAPER, 2 = TISORES
La teva jugada: 0
La màquina ha jugat: TISORES
Has guanyat!
(base) inma@Inmaculadas-MacBook-Pro-2 ex14 % ./main
Joc: 0 = PEDRA, 1 = PAPER, 2 = TISORES
La teva jugada: 2
La màquina ha jugat: TISORES
Heu empatat!
(base) inma@Inmaculadas-MacBook-Pro-2 ex14 % ./main
Joc: 0 = PEDRA, 1 = PAPER, 2 = TISORES
La teva jugada: 89
Jugada no vàlida.
(base) inma@Inmaculadas-MacBook-Pro-2 ex14 %
```

15. Escriu la funció `seguent_color()` que, donat un color d'un semàfor, retorni el color corresponent a l'estat següent: després de VERMELL ve VERD, després de VERD ve GROC i després de GROC torna a VERMELL.

    La funció assumeix que color és un valor vàlid de l'enumeració ColorSemafor.

    Defineix l'enum i la capçalera a **semafor.h**:

```
typedef enum { VERMELL, GROC, VERD } ColorSemafor;

ColorSemafor seguent_color(ColorSemafor color);
```

    I el cos de la funció a **semafor.c**.

Més tard, quan la funció `seguent_color()` estigui completament testejada, implementeu la funció `main()` perquè demani a l'usuari un color (0 = VERMELL, 1 = GROC, 2 = VERD), cridi la funció `seguent_color()` i mostri per pantalla el color següent del semàfor. (funció a **semafor.h** i **semafor.c**, E/S a **main.c**)

Si us plau, segueix el mateix format de els exemples d'execució a continuació:

```
(base) inma@Inmaculadas-MacBook-Pro-2 ex15 % ./main
Color actual del semàfor:
0 = VERMELL
1 = GROC
2 = VERD
Introdueix el color: 0
El següent color és: VERD
(base) inma@Inmaculadas-MacBook-Pro-2 ex15 % ./main
Color actual del semàfor:
0 = VERMELL
1 = GROC
2 = VERD
Introdueix el color: 2
El següent color és: GROC
(base) inma@Inmaculadas-MacBook-Pro-2 ex15 % ./main
Color actual del semàfor:
0 = VERMELL
1 = GROC
2 = VERD
Introdueix el color: 7
Color no vàlid.
(base) inma@Inmaculadas-MacBook-Pro-2 ex15 %
```

6.3. Exercicis de iteratives

A la següent llista d'exercicis es treballen les composicions iteratives, encara que també haureu fer servir les seqüencials i alternatives. NO es poden fer servir ARRAYS.

16. Escriu la funció `sumar_interval()`, que rep un interval tancat `[inici, fi]` i retorna la suma de tots els nombres enters de l'interval. Per exemple, `sumar_interval(1, 5)` ha de retornar `15`, mentre que `sumar_interval(-2, 2)` ha de retornar `0`. La funció assumeix que `inici <= fi`. (funció a **suma_interval.c**)

    ```
    int sumar_interval(int inici, int fi)
    ```

17. Escriu la funció `obtenir_direccio(int inici, int fi)`, que determini en quin sentit s'ha de recórrer l'interval definit pels enters `inici` i `fi`. La funció ha de retornar `1` si cal recórrer l'interval en sentit ascendent (`inici < fi`), `-1` si cal recórrer-lo en sentit descendent (`inici > fi`) i `0` si tots dos valors són iguals. (funció a **descens_ascens.c** asserts a **main.c**)

    ```
    int obtenir_direccio(int inici, int fi)
    ```

    Més tard, quan la funció `obtenir_direccio()` estigui completament testejada, implementeu la funció `main()` perquè demani dos enters a l'usuari, recorri l'interval tancat definit per aquests dos valors en el sentit corresponent (ascendent si `inici < fi` i descendent si `inici > fi`) i mostri tots els nombres per pantalla.

    Per exemple, si l'usuari introdueix `4` i `8`, el programa ha de mostrar: `4 5 6 7 8`

i si introdueix `8` i `4`, ha de mostrar: `8 7 6 5 4`. Si els dos nombres són iguals, només s'ha de mostrar aquest nombre. (funció a **descens_ascens.c**, E/S a **main.c**)

18. Escriu la funció `calcular_max()`, que retorni el més gran dels dos enters `a` i `b`. Si tots dos valors són iguals, la funció ha de retornar qualsevol dels dos. (funció a **max.c**, asserts a **main.c**)

    ```
    int calcular_max(int a, int b)
    ```

    Més tard, quan la funció `calcular_max()` estigui completament testejada, implementeu la funció `main()` perquè demani a l'usuari un enter positiu `n` i, a continuació, una seqüència de `n` enters. El programa ha de determinar el valor màxim de la seqüència fent servir la funció `calcular_max ()` i mostrar-lo per pantalla. La funció `main()` assumeix que `n >= 1`. (funció a **max.c**, E/S a **main.c**)

19. Escriu la funció `total_asteriscs()`, que retorni el nombre total d'asteriscs que contindrà un triangle format per `files` línies, on la primera línia conté un asterisc, la segona dos, i així successivament fins a la línia `files`. Per exemple, si `files = 4`, la funció ha de retornar `10`, ja que el triangle conté `1 + 2 + 3 + 4 = 10` asteriscs. Si `files <= 0`, la funció ha de retornar `0`. (funció a **triangle_asterics.c**, asserts a **main.c**)

    ```
    int total_asteriscs(int files)
    ```

    Més tard, quan la funció `total_asteriscs()` estigui completament testejada, implementeu la funció `main()` perquè demani a l'usuari el nombre de files del triangle, dibuixi el triangle d'asteriscs per pantalla i, finalment, mostri el nombre total d'asteriscs utilitzant la funció `total_asteriscs()`. (funció a **triangle_asterics.c**, E/S a **main.c**)

    Per exemple, si l'usuari introdueix `4`, el programa ha de mostrar:

    ```
    *
    **
    ***
    ****
    ```

20. Escriu la funció `avaluar_polinomi()` que retorni el valor del polinomi amb coeficients `a`, `b`, `c`, per al valor de `x` rebut com a parametre (funció a **polinomi.c,** asserts a **main.c**)

    $$f(x) = ax^2 + bx + c$$

    ```
    double avaluar_polinomi(double a, double a, double c, double x)
    ```

    Més tard, quan la funció `avaluar_polinomi()` estigui completament testejada, implementeu la funció `main()` perquè demani a l'usuari els coeficients `a`, `b` i `c` del polinomi i escrigui per pantalla el valor de la funció per a `x = 0, 1, 2, ..., 20`. (funció a **polinomi.c,** E/S a **main.c**)

21. Escriu la funció `es_casella_blanca()`, que retorni `true` si la casella situada a la posició `(fila, columna)` d'un tauler d'escacs és blanca i `false` si és negra. La funció assumeix que `fila >= 0` i `columna >= 0`. (funció a **escacs.c,** asserts a **main.c**)

    ```
    bool es_casella_blanca(int fila, int columna),
    ```

    Més tard, quan la funció `es_casella_blanca()` estigui completament testejada, implementeu la funció `main()` perquè demani a l'usuari la mida `n` del tauler (`n >= 1`) i dibuixi per pantalla un tauler d'escacs de dimensions `n × n`, representant les caselles blanques amb el caràcter `B` i les negres amb el caràcter `N`. (funció a **escacs.c,** E/S a **main.c**)

    Per exemple, si `n = 5`, la sortida ha de ser:

    ```
    Introdueix la mida del tauler: 5
    Tauler d'Escacs:
    B N B N B
    N B N B N
    B N B N B
    N B N B N
    B N B N B
    ```

22. Escriu un programa que demani a l'usuari un enter positiu `n` (`n >= 1`) i dibuixi per pantalla una graella de dimensions `n × n` on cada casella mostri les seves coordenades `(x, y)`. Les coordenades han de començar a `(0, 0)` a la cantonada inferior esquerra i acabar a `(n-1, n-1)` a la cantonada superior dreta. Fixa't que no es demana cap funció sinò tot el codi a la funció principal (E/S **mapa_coordenadas.c**)

    Per exemple, amb `n=4` la graella és:

    (0, 3) (1, 3) (2, 3) (3, 3)
    (0, 2) (1, 2) (2, 2) (3, 2)
    (0, 1) (1, 1) (2, 1) (3, 1)
    (0, 0) (1, 0) (2, 0) (3, 0)

23. Escriu la funció `suma_xifres()`, que retorni la suma de les xifres del nombre enter `n`.

    ```
    int suma_xifres(int n)
    ```

    Per exemple, `suma_xifres(1234)` ha de retornar `10`, ja que `1 + 2 + 3 + 4 = 10`, i `suma_xifres(9005)` ha de retornar `14`. La funció assumeix que `n >= 0`. (funcio a **xifres.c**, asserts a **main.c**)

24. Escriu la funció `mcd()` que, donat els enters positius `a` i `b`, retorni el màxim comú divisor (MCD). La funció assumeix que `a >= 1` i `b >= 1`. (funció a **mcd.c**, asserts a **main.c**)

    ```
    int mcd(int a, int b)
    ```

    Per calcular el MCD, implementa l'algorisme d'Euclides, que consisteix a repetir el procés següent fins que el segon nombre sigui zero:

    ```
    mentre b ≠ 0
        residu ← a mod b
        a ← b
        b ← residu
    ```

```
fi mentre

retornar a
```

Per exemple, per calcular el MCD de `48` i `18`, aquest procés es repeteix 3 vegades i el resultat és `6`.

25. Escriu la funció `invertir_parelles()` que, donat un enter `n`, retorni un nombre format únicament per les xifres parelles de `n`, mantenint l'ordre invers en què apareixen. La funció assumeix que `n >= 0`. Per exemple: amb `12345678` retorna `8642`, i amb `13579` retorna `0`. Com que la funció retorna un enter, els zeros inicials del resultat no es conserven, per exemple amb 102030 retorna 20. (funció a **invertir.c**, asserts a **main.c**)

```
int invertir_parelles(int n)
```

26. Un tauler de calor és una graella on cada casella conté un valor que representa la seva distància respecte del centre del tauler. Com més a prop del centre es troba una casella, menor és el seu valor. Escriu la funció `calcular_intensitat()` que retorni la distància de Manhattan entre la casella `(fila, columna)` i el centre del tauler, definida com:

$$|\,fila - centre\,| + |\,columna - centre\,|$$

```
int calcular_intensitat(int fila, int columna, int centre),
```

La funció assumeix que fila >= 0, columna >= 0 i centre >= 0. (funció a **intensitat.c**, asserts a **main.c**)

Més tard, quan la funció `calcular_intensitat()` estigui completament testejada, implementa la funció `main()` perquè demani a l'usuari un enter senar positiu n (n >= 1) i dibuixi un tauler de calor de dimensions `n × n`. El centre del tauler correspon a la casella (n/2, n/2). (funció a **intensitat.c**, E/S a **main.c**)

Per exemple, si n = 7, la sortida ha de ser:

```
6 5 4 3 4 5 6
5 4 3 2 3 4 5
4 3 2 1 2 3 4
3 2 1 0 1 2 3
4 3 2 1 2 3 4
5 4 3 2 3 4 5
6 5 4 3 4 5 6
```

27. Un excursionista segueix un pla d'entrenament durant diversos dies. El primer dia camina 1 km, el segon 2 km, el tercer 3 km, i així successivament, però cada 7è dia descansa i no camina cap quilòmetre. Escriu la funció `distancia_tot_entreno()`, que donat el nombre de dies, `diez`, retorni la distància total recorreguda pel excursionista. La funció assumeix que `dies >= 0`. (funció a **excursionista.c**, asserts a **main.c**)

```
int distancia_tot_entreno(int dies)
```

Per exemple, `distancia_tot_entreno(6)` ha de retornar `21`, `distancia_tot_entreno(7)` ha de retornar `21`, i `distancia_tot_entreno(8)` ha de retornar `29`.

28. Un dígit està acompanyat si apareix almenys dues vegades consecutives dins del nombre. Escriu la funció `digits_acompanyats()`, que donat un nombre enter `n>=0`, retorni `true` si tots els dígits del nombre están acompanyats. (funció a **acompanyats.c**, asserts a **main.c**)

```
bool digits_acompanyats(int n)
```

Per exemple, la crida a la funció amb `n 112233` retorna `true`, amb `11122` retorna `true`, amb `12231` retorna `false`, i amb `4556677` retorna `false`

29. En el joc del Nim hi ha una pila de fitxes. Dos jugadors juguen alternativament i, en cada torn, poden retirar 1, 2 o 3 fitxes de la pila (es decideix de forma aleatoria). En aquesta variant del joc, el jugador que retira l'última fitxa perd la partida**.**

    Escriu les funcions `moviment_valid()` i `comprovar_fi_partida()`. (les 2 funcions a **nim.c**, asserts a **main.c**)

    A la funció `moviment_valid()`, `agafades` representa el nombre de fitxes que el jugador vol retirar i `restants` el nombre de fitxes que queden a la pila abans de la jugada. La funció ha de retornar `true` si la jugada és vàlida, és a dir, si el jugador retira entre 1 i 3 fitxes i no n'agafa més de les que queden. La funció assumeix que `restants >= 0`.

    ```
    bool moviment_valid(int agafades, int restants)
    ```

    A la funció `comprovar_fi_partida()`, on `restants` representa el nombre de fitxes que queden a la pila després d'una jugada i `torn_actual` indica el jugador que acaba de fer aquesta jugada (`1` o `2`). La funció ha de retornar `0` si la partida encara no ha acabat, `1` si guanya el jugador 1 o `2` si guanya el jugador 2. La funció assumeix que `restants >= 0` i que `torn_actual` és `1` o `2`.

    ```
    int comprovar_fi_partida(int restants, int torn_actual)
    ```

    Més tard, quan les funcions `moviment_valid()` i `comprovar_fi_partida()` estiguin completament testejades, implementeu la funció `main()` per una partida entre dos jugadors. La partida començarà amb 20 fitxes. Després de cada torn s'haurà de mostrar el nombre de fitxes restants i, quan la partida finalitzi, indicar quin jugador ha guanyat. (les 2 funcions a **nim.c**, E/S a **main.c**)

    Exemple d'execució: (s'indiquen en cursiva i subratllat les entrades per teclat del jugador/a):

    Benvingut al joc del Nim! (Hi ha 20 fitxes, qui agafi l'última perd)

    Queden 20 fitxes. Torn del Jugador 1.
    Quantes fitxes vols agafar (1-3)? 1

    Queden 19 fitxes. Torn del Jugador 2.

Quantes fitxes vols agafar (1-3)? 3

…

Queden 2 fitxes. Torn del Jugador 2.
Quantes fitxes vols agafar (1-3)? 1

Queden 1 fitxes. Torn del Jugador 1.
Quantes fitxes vols agafar (1-3)? 1

La partida ha acabat! Guanya el Jugador 2! Felicitats.

30. En el joc TiraDaus, un jugador és l'ordinador i l'altre l'usuari. Tots dos comencen amb 0 punts. En cada tirada s'obté un valor aleatori entre 1 i 6 (revisa l'apèndix Generar nombres aleatoris), que se suma als punts acumulats. Guanya el jugador que aconsegueix apropar-se més a 13 punts sense superar-los. Si un jugador supera els 13 punts, perd automàticament. Si tots dos superen els 13 punts, no hi ha cap guanyador.

    Escriu les funcions `ordinador_vol_dau()` i `determinar_guanyador_daus(`, (les 2 funcions a **tira_daus.c**, asserts a **main.c**)

    La funció `ordinador_vol_dau()` rep `punts_actuals`, que representa la puntuació acumulada de l'ordinador, i retorna `true` si l'ordinador ha de tornar a tirar el dau i `false` si s'ha de plantar. L'ordinador segueix l'estratègia de tirar un altre dau sempre que la diferència entre 13 i els seus punts sigui superior a 3 (`>3`).

    ```
    bool ordinador_vol_dau(int punts_actuals)
    ```

    La funció `determinar_guanyador_daus()`, rep `punts_ord`, que representa la puntuació final de l'ordinador i, `punts_usr` que representa la puntuació final de l'usuari. La funció ha de retornar `1` si guanya l'ordinador, `2` si guanya l'usuari o `0` si no hi ha cap guanyador. En cas que cap jugador superi els 13 punts, guanya qui tingui més punts. En cas d'empat, es considera que guanya l'ordinador.

    ```
    int determinar_guanyador_daus(int punts_ord, int punts_usr)
    ```

    Més tard, quan les funcions `ordinador_vol_dau()` i `determinar_guanyador_daus()` estiguin completament testejades, implementeu la funció `main()` perquè simuli una partida del joc. L'ordinador jugarà primer utilitzant l'estratègia indicada. A continuació serà el torn de l'usuari, que decidirà després de cada tirada si vol tornar a tirar el dau (`S`) o plantar-se (`N`). Al final de la partida s'haurà de mostrar el guanyador. (les 2 funcions a **tira_daus.c**, E/S a **main.c**)

    Exemple d'execució: (s'indiquen en cursiva i subratllat les entrades per teclat del jugador/a)

    Torn de l'ordinador.
    0 + 1 = 1
    1 + 5 = 6
    6 + 2 = 8
    8 + 4 = 12
    L'ordinador es planta amb 12 punts.

    Torn de l'usuari.

```
0 + 1 = 1
Tirar dau (S/N)? s
1 + 5 = 6
Tirar dau (S/N)? s
6 + 3 = 9
Tirar dau (S/N)? s
9 + 5 = 14
L'usuari es passa amb 14 punts.

Guanya l'ordinador amb 12 punts!
```

31. Dos cangurs avancen simultàniament fent salts sobre una línia recta. El primer cangur parteix de la posició `x1` i fa salts de longitud `s1`. El segon cangur parteix de la posició `x2` i fa salts de longitud `s2`.

    Escriu la funció `es_troben()`, on `x1` i `x2` representen les posicions inicials dels dos cangurs, i `s1` i `s2` les longituds dels seus salts. La funció ha de retornar `true` si, després d'un mateix nombre de salts, els dos cangurs coincideixen en una mateixa posició, i `false` en cas contrari. La funció assumeix que `x1 >= 0`, `x2 >= 0`, `s1 > 0` i `s2 > 0`. **(**les funció a **cangur.c,** asserts a **main.c)**

    ```
    bool es_troben(int x1, int s1, int x2, int s2)
    ```

32. Escriu la funció `es_senar()`, on `num` representa un nombre enter. La funció ha de retornar `true` si `num` és senar i `false` en cas contrari.

    ```
    bool es_senar(int num)
    ```

    Escriu la funció `calcular_mitjana()`, on `suma_total` representa la suma d'una serie de valors i `quantitat` el nombre de valors. La funció ha de retornar la mitjana dels nombres. Si `quantitat` és `0`, la funció ha de retornar `0.0`. La funció assumeix que `quantitat >= 0`.

    ```
    double calcular_mitjana(int suma_total, int quantitat)
    ```

    (les funcions a **mitjana_senars.c,** asserts a **main.c)**

    Més tard, quan les funcions `es_senar()` i `calcular_mitjana()` estiguin completament testejades, implementeu la funció `main()` perquè demani a l'usuari un enter positiu `n` (`n >= 1`), llegeixi a continuació `n` nombres enters, calculi la mitjana dels nombres senars introduïts i la mostri per pantalla. Si no s'ha introduït cap nombre senar, el programa haurà de mostrar "No s'han introduït nombres senars.". **(**les funcions a **mitjana_senars.c,** E/S a **main.c)**

33. En el joc d'endevinar un nombre, l'ordinador genera aleatòriament un nombre enter dins del rang `[0, 5]`. El jugador ha d'intentar endevinar-lo.

    Escriu la funció `es_rang_valid()`, on `num` representa el nombre introduït pel jugador. La funció ha de retornar `true` si `num` pertany al rang `[0, 5]` i `false` en cas contrari. La funció no assumeix cap precondició.

    ```
    bool es_rang_valid(int num)
    ```

Escriu la funció `avaluar_intent()`, on `secret` representa el nombre secret generat per l'ordinador i `intent` el nombre introduït pel jugador. La funció ha de retornar `0` si el jugador ha encertat el nombre secret, `1` si el nombre secret és major que l'intent i `-1` si el nombre secret és menor que l'intent. La funció assumeix que `secret` i `intent` són valors compresos entre `0` i `5`.

```
int avaluar_intent(int secret, int intent)
```

(les funcions a **endevinar_nombre.c,** asserts a **main.c)**

Més tard, quan les funcions `es_rang_valid()` i `avaluar_intent()` estiguin completament testejades, implementeu la funció `main()` perquè simuli una partida del joc. El programa haurà de generar aleatòriament el nombre secret, demanar intents al jugador fins que l'encerti, informar després de cada intent si el nombre secret és major o menor i, en acabar la partida, preguntar si es vol tornar a jugar (`s/n`). (les funcions a **endevinar_nombre.c,** E/S a **main.c**)

Exemple d'execució: (s'indiquen en cursiva i subratllat les entrades per teclat del jugador/a)

```
Benvingut/da al joc d'endevinar nombres! Estic
pensant un nombre entre 0 i 5....
Quin nombre creus que he pensat?:
9
El nombre 9 no és vàlid. Ha d'estar en el rang [0, 5]. Estic
pensant un nombre entre 0 i 5....
Quin nombre creus que he pensat?:
2
El nombre secret era 1, és menor que 2. Estic
pensant un nombre entre 0 i 5.... Quin
nombre creus que he pensat?:
0
El nombre secret era 4, és major que 0. Estic
pensant un nombre entre 0 i 5.... Quin
nombre creus que he pensat?:
3
Has esbrinat que era 3! Has guanyat! Vols
jugar altre vegada? [s/n]
n
Fins altra vegada!
```

7. Apèndix

7.1. Generar nombres aleatoris

Per generar nombres aleatoris en C s'utilitza la funció `rand()`, definida a `<stdlib.h>`. `rand()` genera un nombre enter pseudoaleatori comprès entre 0 i RAND_MAX (ambdós inclosos), on RAND_MAX és una constant definida per la implementació del compilador.

Abans de fer-la servir, cal inicialitzar la llavor (seed) del generador amb `srand(time(NULL))` (dela llibreria <time.h>), normalment al començament de `main()`, perquè cada execució del programa generi una seqüència diferent de nombres.

Per obtenir un nombre enter entre un mínim i un màxim (ambdós inclosos), s'utilitza la fórmula:

```
int n = rand() % (maxim - minim + 1) + minim;
```

Per exemple, `rand() % 10` genera un nombre entre 0 i 9, mentre que `rand() % 11 + 5` genera un nombre entre 5 i 15. Codi exemple a continuació:

```c
#include <stdio.h>
#include <stdlib.h> //per srand
#include <time.h>  //per time

int main(void) {
    int aleatori, minim, maxim, n;
    // inicialitza el generador de nombres aleatoris
    srand(time(NULL));

    minim = 5;
    maxim = 15;

    // Genera un nombre pseudoaleatori
    n = rand();
    printf("Valor retornat per rand(): %d\n", n);
    printf("Valor de RAND_MAX: %d\n", RAND_MAX);

    // genera un nombre aleatori entre minim i maxim (ambdós inclosos)
    aleatori = rand() % (maxim - minim + 1) + minim;

    printf("Nombre aleatori entre %d i %d: %d\n", minim, maxim, aleatori);

    return 0;
}
```

Compilació i execució del codi anterior:

```
(base) inma@Inmaculadas-MacBook-Pro-2 GeneracioAleatoris % gcc -Wall -std=c17 -pedantic aleatoris.c -o aleatoris
(base) inma@Inmaculadas-MacBook-Pro-2 GeneracioAleatoris % ./aleatoris
Valor retornat per rand(): 1876824361
Valor de RAND_MAX: 2147483647
Nombre aleatori entre 5 i 15: 8
(base) inma@Inmaculadas-MacBook-Pro-2 GeneracioAleatoris % ./aleatoris
Valor retornat per rand(): 1876857975
Valor de RAND_MAX: 2147483647
Nombre aleatori entre 5 i 15: 10
(base) inma@Inmaculadas-MacBook-Pro-2 GeneracioAleatoris %
```